Observed Recoverable Behavioral Failure Modes in LLM Workflows

A Multi-Session Cross-Platform Case Study, Retry-Probe Pilot, and Multi-Model Evaluation Protocol

Vijay Suresh · Version 1.8 · May 2026

Contents

Abstract

Standard large-language-model evaluations primarily score terminal outputs: whether a final answer is correct, safe, preferred, or useful. This paper studies a different class of failures, which we call *recoverable behavioral failures* (RBFs). An RBF occurs when a model appears to possess the capability, context, and tool access needed to complete a task but does not exercise that capability on the first pass, and subsequently succeeds after a brief retry, verification, or correction prompt that introduces no new substantive information. RBFs are observable in long, practical AI workflows including research synthesis, source verification, document drafting, code generation, multimodal interpretation, and artifact production. They are not captured by single-turn benchmark accuracy or by aggregate human preference data, because they only become visible across turns, when a second prompt reveals that the underlying capability was already available.

The paper makes five contributions. First, it defines recoverable behavioral failure as a distinct evaluation construct, separating capability from first-pass execution behavior. Second, it develops a layered sixteen-mode taxonomy spanning retrieval, reasoning, multimodal interpretation, infrastructure, artifact production, and editorial boundary control. Third, it reports a retry-probe pilot on a fifteen-item heterogeneous retrieval task in which first-pass resolution was 12 / 15 and after-retry resolution was 15 / 15, yielding a 20 percentage-point recoverability gap. Wilson 95% confidence intervals are reported for the small-sample rates, and an exact small-sample Fisher test associates first-pass failure with one structural item class (two-sided $p = 0.0022$, exact). Fourth, it extends the taxonomy with abstracted cross-platform workflow observations covering artifact-readiness, source-verification, audience-calibration, and editorial-boundary modes. Fifth, it specifies a controlled multi-model evaluation protocol grounded in a published task pack, with task families, deterministic and human-judged checks, sample-size guidance, an annotation rubric, and a reproducibility checklist. The pilot is exploratory; the multi-model protocol is offered as a planned replication rather than a completed study.

Keywords: large language models; evaluation; behavioral failures; retry behavior; agent reliability; tool use; artifact generation; self-verification; workflow readiness; case-study methodology; reproducible benchmark.

1. Introduction

Public discussion of language-model failures has converged on two broad categories. A *capability failure* occurs when the model lacks the knowledge, reasoning skill, perceptual ability, or tool fluency required to solve the task. An *alignment or safety failure* occurs when the model produces output that is harmful, deceptive, or otherwise off-policy. Existing evaluation infrastructure is built around these two categories: benchmark accuracy [Liang et al., 2022; Srivastava et al., 2023; Hendrycks et al., 2021] indexes the first; preference-based reinforcement learning [Christiano et al., 2017; Bai et al., 2022] and red-teaming index the second.

Many failures observed in practical AI workflows fit neither category cleanly. A user issues a request; the model declares part of the task unresolved, incomplete, or impossible; a short follow-up prompt elicits a substantively correct response without introducing new documents, tools, or facts. The capability was demonstrably available on the first turn. What changed between attempts was a search strategy, a verification step, a recognition of structural input, or an audit of an artifact for placeholders, formatting, or boundary leakage. The first attempt did not fail because the model could not do it. It failed because the model did not.

We refer to these events as *recoverable behavioral failures*. They are *behavioral* not in any anthropomorphic sense but because the failure is located in execution policy: effort allocation, persistence, verification,

escalation, search discipline, multimodal reading, artifact finalization, and boundary management between conversation and deliverable. They are *recoverable* because a local correction reveals the model was capable of producing the correct output without additional information.

The distinction matters for modern AI workflows. Frontier models are now used for research synthesis, software debugging, document drafting, slide and spreadsheet generation, literature triage, citation checking, and tool-assisted operations. These are not single-turn tasks with a single final answer. They are multi-step, supervised processes in which the practical cost of a model is not only the probability of a wrong terminal answer; it is the amount of human oversight needed to detect premature stopping, recover incomplete artifacts, correct unsupported claims, and prevent process commentary from contaminating the final deliverable.

This paper is intentionally conservative. It does not estimate population-level failure rates for any vendor, model family, or product. It develops the construct, codes a naturalistic multi-session corpus, measures one mode quantitatively in a small pilot, extends the taxonomy to artifact-production and editorial-boundary failures observed in cross-platform professional workflows, and specifies a multi-model evaluation protocol that a future study can run to replicate and extend the findings.

1.1 Contributions

1. **Construct.** A definition of recoverable behavioral failure that separates capability from first-pass execution behavior, with operational measurement instruments (first-pass success, retry success, recoverability gap).

2. **Taxonomy.** A layered sixteen-mode taxonomy organized by failure locus (model/retrieval policy, reasoning, multimodal interpretation, artifact generation, client/system, evaluation pipeline).

3. **Pilot.** A retry-probe pilot with first-pass rate 12 / 15 = 80% (Wilson 95% CI: [54.8%, 93.0%]), retry rate 15 / 15 = 100%, and recoverability gap +20 percentage points (CI on discordant proportion: [7.0%, 45.2%]). Exact Fisher test on item-structural class association gives two-sided $p = 0.0022$.

4. **Cross-workflow extensions.** Four additional modes derived from professional artifact-production and public-communication workflows: artifact-finalization failure, source-verification drift, audience/context miscalibration, and editorial boundary contamination.

5. **Multi-model protocol.** A controlled benchmark specification with task families, deterministic and human-judged checks, sample-size guidance, annotation rubric, and reproducibility package, grounded in an accompanying ten-task pilot pack covering five task categories.

2. Related Work

2.1 LLM benchmarks and evaluation

Holistic Evaluation of Language Models [Liang et al., 2022], BIG-Bench [Srivastava et al., 2023], and MMLU [Hendrycks et al., 2021] established broad coverage of LLM capabilities across reasoning, knowledge, and language tasks. Their primary measurement target is the correctness, calibration, or preference of a terminal output under a fixed prompt. These benchmarks are essential and continue to drive frontier progress. They do not, however, separate the gap between what a model produces on the first attempt and what it can produce after an explicit retry, verification, or alternative search strategy. The present paper complements terminal-output evaluation by treating first-pass and post-retry performance as separately measurable quantities and by introducing artifact-level readiness checks alongside text-level correctness.

2.2 RLHF and preference learning

Reinforcement learning from human feedback [Christiano et al., 2017; Bai et al., 2022] and subsequent work on direct preference optimization aggregate pairwise human preferences over candidate responses. Preference labels are informative about which of two final answers is preferred, but they do not by themselves encode the reason for the preference. If the dispreferred answer would have improved under a follow-up retry without new information, that recoverability is not captured in the preference signal. As a result, a model trained to optimize aggregate preference can remain susceptible to behavioral failures whose cost is paid in turns of supervision rather than in measured preference deltas.

2.3 Tool-use and agentic benchmarks

SWE-bench [Jimenez et al., 2024] and WebArena [Zhou et al., 2024] evaluate complex multi-step tasks in software-engineering and web-navigation environments respectively. These agentic benchmarks measure end-to-end task completion under tool access. They do not always separate the intermediate decisions that constitute the agent's policy: when to retry a thin search, when to escalate to a different query construction, when to verify a numerical claim, when to ask for structural confirmation of a multimodal input, and when to declare an artifact complete. Recoverable behavioral failures are visible precisely at those intermediate decisions, which is why this paper introduces both retry-probe metrics and artifact-readiness checks.

2.4 Truthfulness, sycophancy, and deception

TruthfulQA [Lin et al., 2022] targets one class of behavioral failure: the assertion of false-but-popular claims. Work on sycophancy [Sharma et al., 2023] documents the tendency to align responses with stated user preferences even when those preferences diverge from the correct answer. Park et al. [2024] survey AI deception and adjacent confidently-wrong behavior. The present taxonomy intersects this literature but extends to a complementary surface pattern: the model may also decline, truncate, misread, or produce an incomplete artifact rather than confidently assert a false proposition. These behaviors can be equally costly in real workflows even when they do not look like classical hallucination.

2.5 Time-sensitive QA and knowledge staleness

Time-sensitive question answering [Chen et al., 2021; Liska et al., 2022] documents that static model knowledge degrades as the world changes. The present paper extends this concern from factual decay to procedural decay. When a model confidently describes a third-party tool interface that has been restructured, the resulting failure is not a factual hallucination in the traditional sense; it is a stale procedural assumption that wastes user effort. The proposed mitigation, confidence-decay framing, is the procedural analogue of well-known calibration strategies on factual outputs.

2.6 Long-horizon agent reliability and human-AI workflow reliability

A growing body of empirical and observational work documents the gap between single-turn capability and multi-turn reliability. Agent benchmarks report success rates that decline monotonically with task length, number of tool calls, and number of subgoals, even when single-step accuracy on the same operations is high. Studies of human-AI collaboration on knowledge-intensive tasks document substantial supervisory load placed on users when assistants fail in recoverable but uncaught ways: users absorb the cost of detecting premature stopping, of re-issuing corrective prompts, and of auditing artifacts whose surface form looks acceptable. Work on writing-assistance, coding-assistance, and research-assistance settings repeatedly finds that the time savings attributable to AI assistance are partly recouped by the time spent verifying outputs the assistant produced with insufficient self-checking.

The construct of recoverable behavioral failure offers a unified measurement language for these observations. Each subgoal in a long workflow is itself a candidate first-pass decision: the agent may search, verify, escalate, finalize, or audit, and may do so well or poorly. The practical reliability of the workflow is the cumulative product of those decisions. Reporting first-pass and retry-after-prompt rates separately, and augmenting with artifact-readiness and boundary-contamination checks, converts the informal supervisory-load observation from human-AI collaboration into a set of measurable rates that can be tracked over time and compared across models.

2.7 Case-study methodology

The empirical structure of this paper follows the theory-building case-study logic in Yin [2018]. Naturalistic cases are appropriate when the phenomenon is multi-turn, workflow-dependent, and not yet reducible to isolated benchmark questions. The role of the case study is not to replace controlled experimentation. It is to discover and characterize constructs that can then be measured through controlled replication. The retry-probe pilot in §7 and the multi-model protocol in §9 implement that subsequent measurement step.

3. Conceptual Framework

We define a recoverable behavioral failure relative to a verifiable task. Let an item i in task set T have a verifiable target outcome y_i. Let $A_1(i)$ denote the model's **first-pass** attempt under the original prompt, and let $A_r(i)$ denote the model's **retry** attempt after a standardized retry or recovery prompt that supplies no new substantive information—no new documents, no new tools, no new facts, no new task specification, and no relaxed evaluation standard. Let $S_1(i) = 1$ when $A_1(i)$ satisfies the verification criterion, and analogously $S_r(i)$ for the retry attempt.

3.1 Core quantities

- **First-pass success rate:** $p_1 = (1 / N) \cdot (\text{sum over items i of } S_1(i))$.
- **Retry success rate:** $p_r = (1 / N) \cdot (\text{sum over items i of } S_r(i))$.
- **Recoverability gap:** $G = p_r - p_1$.
- **Recoverable behavioral failure indicator** for item i: equal to 1 if and only if $S_1(i) = 0$ and $S_r(i) = 1$ under the standardized retry.

The indicator captures items where the first attempt failed and the retry succeeded under a prompt that supplied no new substantive information. Aggregating the indicator across items yields the empirical RBF rate; aggregating across items and conditions yields the recoverability gap.

3.2 Construct boundaries

The construct is intentionally conservative. The following are **not** counted as recoverable behavioral failures, even though they may resemble them on a surface read:

- Retries in which the user supplies a missing fact, document, or answer (ordinary interactive correction).
- Retries in which the task specification or evaluation standard is relaxed (specification slippage).
- Retries in which new tool affordances become available between attempts (tool unlock).
- Cases where the first-pass output is correct under a reasonable reading and the user demands an unnecessary rephrase (preference rework).

These exclusions protect the construct from inflation. Without them, every multi-turn dialogue would appear to contain behavioral failures. With them, the construct captures a specific phenomenon: the model demonstrably had the capability and context to succeed on the first pass and did not.

3.3 Additional measurable constructs

- **Batch-scaling degradation.** The change in per-item first-pass success as batch size N varies.
- **Artifact-readiness failure.** The model produces a deliverable that nominally exists but does not meet user-facing readiness criteria.
- **Boundary-contamination failure.** Process commentary, advisory metadata, or chat-only content appears inside a deliverable that should be publication-ready.
- **Verification compliance.** The proportion of items for which the model performs a stated verification step when the task implies one is needed.

3.4 Summary of core constructs

Construct	Operational definition	Primary observable
First-pass success	Task solved under the original prompt	Correct final answer or artifact on first attempt
Retry success	Task solved after standardized retry with no new info	Correct answer or artifact after retry such as 'try a different approach'
Recoverability gap	Difference between retry and first-pass success rates	$p_r - p_1$
Recoverable failure	$S_1(i) = 0$ and $S_r(i) = 1$ under standardized retry	Item-level indicator
Premature termination	Declares item unresolved despite available alternatives	Unknown on first pass; resolved after escalation
Artifact readiness	Deliverable satisfies user-facing completion criteria	Opens, formats, contains no placeholder or boundary contamination
Editorial-boundary control	Keeps process commentary out of publishable artifact	No meta-conversation content inside final deliverable

Table 1. Core constructs used to measure recoverable behavioral failures.

4. Data and Methodology

4.1 Evidence base

The evidence combines two sources. The first is a fourteen-session naturalistic case study of multi-turn workflows in document-generation, source-verification, multi-step research, quantitative reasoning, structured-visual interpretation, and publication-preparation contexts. The second is a cross-platform abstraction over additional professional workflows involving artifact production and public-communication preparation. Underlying raw conversations include personal and work-adjacent material; for this manuscript the paper reports only abstracted workflow categories and failure mechanisms, with no individual identifying details.

4.2 Evidence tiers

To prevent overstating the strength of any individual claim, observations are assigned to evidence tiers. Claims throughout the paper are framed according to the tier of their supporting evidence.

Tier	Evidence type	Use in paper	Constraints
Tier 1	Measured pilot item outcomes	Quantitative PTS measurement and recoverability gap	Small N; one task batch; exploratory statistics
Tier 2	Transcript-verifiable multi-turn observations	Mode definitions and worked examples	Underlying transcripts not publicly released
Tier 3	Abstracted cross-platform workflow summaries	Artifact and editorial-boundary extensions	Summary-level evidence; not a full raw export
Tier 4	Hypothesized mechanisms with proposed tests	Experimental design and future benchmark	Not reported as completed results

Table 2. Evidence tiers used to separate measured results from observational and proposed claims.

4.3 Coding procedure

Each candidate event is coded with the following fields: (a) task context; (b) original model output or artifact; (c) user correction or retry prompt; (d) post-correction outcome; (e) whether new substantive information was introduced; (f) failure mode; (g) trigger; (h) recovery type; (i) user-visible cost. A conservative coding rule applies: if recovery requires new information from the user, the event is excluded from recoverable behavioral failures. Such events may still be informative as ordinary interactive corrections, but they are not counted toward the construct.

4.4 Model-attribution caveats

The Claude conversation export schema does not record per-message model identity. Model attribution in the original case study is reconstructed from user recall and contextual cues. Because this reconstruction is incomplete, the present paper avoids strong model-comparison claims grounded in the case study. The proposed multi-model protocol in §9 is the appropriate vehicle for controlled model comparison, since it administers matched tasks across model families under standardized prompts.

4.5 Responsible data handling

Because conversational AI histories can include sensitive personal, professional, and third-party information, the present manuscript does not release raw transcripts. The observation bank that informs the taxonomy is maintained privately. The accompanying benchmark package contains items synthesized from neutral domains; it does not incorporate user-specific identifiers. Future public releases should follow a two-layer model: a public benchmark designed from neutral sources, with prompts and expected outcomes; and a private audit appendix for trusted reviewers, with sensitive details redacted.

5. Taxonomy of Recoverable Behavioral Failures

The taxonomy is layered by failure locus. Locus separation matters because not every user-visible failure should be attributed to the model core. Some failures arise in retrieval policy; some in batch-effort allocation; some in artifact generation; some in client transport; some in product memory; some in the evaluation pipeline itself. A useful framework should locate the failure precisely enough to guide the right mitigation.

5.0 A note on mode overlap and granularity

Sixteen modes is a deliberate choice. Several pairs of modes are clearly adjacent and could in principle be collapsed: PTS and SMI are both retrieval-policy failures; MRT, UCT, and MSI are all interruption-style failures at different loci; AFF and EBC are both artifact-side failures. We keep them separate in this taxonomy for three reasons. First, the *mitigation* implied by each mode is different (escalation policy versus consistency policy; resume-state preservation versus rate-limit handling; readiness audit versus boundary audit); collapsing the codes would obscure the mitigation taxonomy. Second, the *locus* attribution is different across the apparent pairs (model retrieval policy versus client transport versus rate limiter); collapsing them would obscure where instrumentation is needed. Third, an empirically-driven collapse should follow from data: once cross-model and cross-user data accumulates, modes that always co-occur in the same items can be merged with reviewer confidence, while modes that dissociate empirically should stay separate. The present taxonomy is therefore offered as a starting partition for measurement, not as a final structural claim.

Code	Name	Primary locus	User-visible cost
PTS	Premature Termination of Search	Model / retrieval policy	False unresolved answers; missed information
BDTF	Breadth-Depth Tradeoff Failure	Model policy under batch load	Shallow per-item analysis presented as complete coverage
SMI	Search Methodology Inconsistency	Retrieval strategy	Inconsistent escalation across similar subtasks
MRT	Mid-Response Truncation	Infrastructure / model interaction	Partial output, lost progress, incomplete generation
MSI	Mobile Streaming Interruption	Client / transport	Dropped response on screen-lock or mobile interruption
FPG	Feedback Persistence Gap	Product / training pipeline	Same corrections not preserved across sessions
RFI	Retry-Failure Invisibility	Evaluation pipeline	Offline review cannot distinguish cannot-solve from did-not-try
STK	Stale Third-Party Tool/UI Knowledge	Model knowledge / tool guidance	Confident instructions for changed live interfaces
QI	Quantitative Inconsistency	Reasoning / self-audit	Contradictory numbers within one response
PPG	Phantom Placeholder Persistence	Artifact / code generation	Template placeholders emitted as executable commands
UCT	Usage-Cap Termination	System / rate limit	Mid-task interruption and degraded resume state
VMC	Visual Misreading Cascade	Multimodal interpretation	Repeated confident misreadings of structured visual input
AFF	Artifact Finalization Failure	Artifact generation	File exists but is incomplete, poorly formatted, or not apply-ready
SVD	Source Verification Drift	Citation / retrieval practice	Claims or links not sufficiently verified against source
EBC	Editorial Boundary Contamination	Artifact boundary management	Process notes, ratings, or strategy inserted into publishable artifact
ACM	Audience / Context Miscalibration	Communication / register	Technically correct content unsafe or off-register for the intended audience

Table 3. Layered taxonomy of observed and proposed recoverable behavioral failure modes.

5.1 Premature Termination of Search (PTS)

Definition. The model declares an information item unresolved after a small number of retrieval attempts, even though a readily available alternative retrieval strategy would resolve it. The first attempt typically uses a thin or generic query construction; the second attempt, issued under a retry prompt that introduces no new information, succeeds.

Observed pattern. In the pilot retrieval task, three items whose URLs contained opaque numeric identifiers were marked unresolved on the first pass after generic keyword queries returned thin results. After a standardized retry prompt the model used site-targeted query construction and resolved all three. The capability gap on the first pass was therefore a query-construction policy decision rather than a knowledge or tool limit.

Why it matters. PTS is among the most consequential modes because it masquerades as missing knowledge. The user receives an ‘unknown’ response and reasonably treats it as a capability boundary. In fact the system has not exhausted available retrieval policy. The cost is paid both as missed information and as user-side supervisory load: the user must recognize that the response is recoverable and issue the retry themselves.

Differentiation. PTS is distinct from **SMI** (which concerns inconsistency across similar subtasks rather than premature stopping on one subtask) and from **RFI** (which is the evaluation-pipeline blind spot that makes PTS invisible in offline review). PTS is also distinct from genuine retrieval failure where no alternative strategy would help; the diagnostic for PTS is that an alternative strategy demonstrably exists.

Detection. Retry probes constructed so that the first-pass query is likely to be thin but at least one alternative strategy is known to succeed. Measure first-pass success and retry-after-prompt success separately; the gap quantifies PTS. Probe construction recipes include items with opaque-identifier URLs, items whose canonical answer is on a specific known site, and items that require an exact-string match the generic query would dilute.

Mitigation. An explicit retrieval escalation policy: when the first search returns thin, the model should systematically try query rewriting, site restriction, exact-string identifier search, and source triangulation before declaring failure. The escalation should be visible to the user so that supervision is calibrated rather than always assumed; this also helps the user trust the ‘unknown’ verdicts that remain after escalation.

5.2 Breadth-Depth Tradeoff Failure (BDTF)

Definition. When presented with a large batch, the model implicitly optimizes for covering all items rather than verifying each item deeply. The result is a superficially complete response with weak per-item grounding: every item appears addressed, but per-item depth degrades as the batch size grows.

Observed pattern. Multi-item evaluation tasks elicit short, uniform dispositions even when individual items differ materially in difficulty. Items requiring secondary lookup are sometimes resolved without the lookup having occurred. Items requiring verification are sometimes presented with confident framings derived from partial reads. The user infers uniform depth from uniform output length, which is exactly the inference the model has not earned.

Why it matters. BDTF is an allocation error in which the appearance of coverage substitutes for verification. The user reads a list, sees that every item has been addressed, and reasonably assumes uniform depth. Without explicit depth indicators the user is forced to re-verify each item to recover trust—at which point the breadth advantage of the batch was illusory and the depth cost has merely shifted.

Differentiation. BDTF differs from **MRT** (which is truncation within a single response) and from **PTS** (which is premature stopping on a single retrieval). BDTF is a batch-level allocation phenomenon: per-item correctness degrades as N grows even when per-item tasks at small N are reliably solved. It can co-occur with PTS but is operationally separable through batch-size variation.

Detection. Batch-scaling curves: administer the same items at batch sizes N = 1, 5, 15, 30. Measure per-item correctness against N. The slope quantifies BDTF; the inflection point identifies the effective batch capacity. A healthy policy under load surfaces verification depth per item rather than hiding allocation behind uniform output length.

Mitigation. Batch-aware effort budgeting: for large batches, the model should state per-item uncertainty, allocate verification depth, and surface items that received only shallow processing. A simple product affordance is per-item flags that distinguish ‘verified,’ ‘partially verified,’ and ‘not verified’ outputs.

5.3 Search Methodology Inconsistency (SMI)

Definition. The model uses stronger retrieval tactics for some items and weaker tactics for similar items within the same session, with no explicit reason for the difference. Query construction varies between generic keyword search, site-targeted search, and identifier-anchored search without a stated escalation rule.

Observed pattern. Across a batch of retrieval subtasks, items that would have benefited from the stronger tactic receive the weaker tactic; users notice the inconsistency only when they observe that one item succeeded under exactly the strategy that was not applied to a similar item. The same model can exhibit PTS on one item, escalate appropriately on a similar item, and fail to apply the same escalation to a third.

Why it matters. Inconsistent retrieval discipline shifts the burden of escalation onto the user. A robust workflow expects a predictable escalation protocol on thin first results: query rewriting, site restriction, identifier search, title extraction, source triangulation. SMI occurs when this escalation is selectively applied, so the user cannot rely on uniform retrieval policy and must monitor strategy choice item by item.

Differentiation. PTS is a failure mode within a subtask; SMI is a consistency mode across subtasks. The two can co-occur but are operationally separable: SMI persists even when overall PTS incidence is low, because the issue is the conditional probability of strategy escalation given a thin first result, not the absolute incidence of premature stopping.

Detection. Per-subtask query logging within a session. Compute (a) the distribution of queries-per-subtask, (b) the distribution of query-construction strategies, and (c) the conditional probability of switching strategy given a thin first result. A healthy retrieval policy should exhibit consistent escalation across structurally similar subtasks.

Mitigation. A documented escalation protocol applied uniformly across retrieval subtasks. Internal logging that exposes the chosen strategy to evaluators and, where appropriate, to the user, so that inconsistencies are noticeable rather than hidden. Consistency is itself a user-trust signal even when the underlying strategy is imperfect.

5.4 Mid-Response Truncation (MRT)

Definition. The model begins generating a response, makes partial progress, and the stream terminates before completion, often after partial tool use or partial drafting. The termination is signaled by a generic error rather than a graceful wind-down.

Observed pattern. Long multi-part responses on heterogeneous batches sometimes terminate with a generic recovery message. Partial work is not always preserved or replayable. The user is required to issue a fresh prompt to resume, with no native carry-over of intermediate state. In long workflows the cost of lost intermediate progress compounds: each truncation imposes recovery work proportional to task complexity.

Why it matters. MRT is costly not only because the answer is incomplete but because the partial work may need to be reassembled manually. The user must reconstruct what the model had already done, decide what to keep, and re-prompt for the remainder. The cost scales with response length and tool-call density—exactly the regime where users were relying on the model to reduce supervisory load.

Differentiation. MRT is an infrastructure-side termination of a single response; **UCT** is a system-level rate-limit termination of the conversation; **MSI** is a client-side stream drop. All three share the user-visible appearance of an interrupted task but differ in locus, signal, and recovery behavior.

Detection. Near-budget telemetry: log the fraction of token, tool-call, or wall-clock budget consumed at response termination. For a healthy generator, the distribution should be bimodal—very early when the task is short, well below threshold when the model voluntarily concluded. A spike of terminations at or near the budget boundary indicates MRT.

Mitigation. A structured progress ledger that allows the model and the system to preserve and resume intermediate state when generation fails. Resume-state checkpoints during long generations, with the option to surface and replay partial outputs. Where graceful termination is feasible, the model should be cued to summarize completed work before the budget is exhausted.

5.5 Mobile Streaming Interruption (MSI)

Definition. Mobile or client-side interruptions can drop an in-progress streaming response, with no recovery of the partial output when the user returns to the session. Screen-lock, backgrounding, and connection drops on mobile clients are common triggers.

Observed pattern. Screen-lock during streaming generation on mobile clients sometimes terminates the connection. The session reopens without the partial response surfaced. Users describe the failure as workflow unreliability rather than as a model error—they may not know whether the model finished and the partial output was lost in transit, or whether the model itself never finished.

Why it matters. Long AI workflows increasingly depend on streaming responses. When the client loses partial output, the cost is workflow-level: the user must re-issue the request and absorb the full latency again. The failure compounds with MRT and UCT to make long sessions on mobile materially less reliable than on desktop.

Differentiation. MSI is client/transport-layer; MRT is infrastructure/model-side; UCT is system rate-limit. All three are recoverable in principle through different mechanisms—client-side buffering for MSI, server-side resume for MRT, graceful continuation for UCT—but the locus determines the correct mitigation.

Detection. Client-side telemetry on stream-drop events correlated with foreground/background lifecycle and with screen-lock events. The relevant rate is interrupted streams per long-generation request on mobile clients, segmented by client version and connection type.

Mitigation. Persistent server-side response buffering with client reattachment on resume, so a screen-lock event does not erase work already produced. Notification surfaces that allow the user to recover the interrupted response on next session open, with the partial output highlighted so the user can decide whether to continue or restart.

5.6 Feedback Persistence Gap (FPG)

Definition. Within-session corrections, scope clarifications, and explicit user preferences do not persist beyond the session unless an explicit memory surface or product mechanism captures them. The same user encounters the same correctable issue in subsequent sessions.

Observed pattern. A user corrects a stylistic or factual error in one session; the same error recurs in subsequent sessions without the correction being applied. Aggregate signal from preference data eventually reaches model training, but individual users observe no compounding return on their corrective effort across sessions.

Why it matters. FPG is not always a defect; user control and privacy require limits on cross-session persistence. The evaluation issue is that repeated corrections impose longitudinal cost and are invisible in single-session benchmarks. The same user who invests heavily in shaping behavior receives the least cumulative benefit absent an explicit memory mechanism—an incentive misalignment for power users.

Differentiation. FPG is the product/training-pipeline analogue of within-session retry behavior. It is distinct from FPG-adjacent caching mechanisms because it concerns user-corrective signal, not retrieval reuse. It is also distinct from explicit memory features whose presence renders FPG inapplicable for the affected facts.

Detection. Out of scope for single-session response-level evaluation. The appropriate measurement is longitudinal: time-to-recurrence of a corrected error, and the proportion of stable preferences that survive across sessions for a given user. These metrics require user-level instrumentation and informed consent.

Mitigation. Explicit per-user memory surfaces with user-visible controls. Treating correction events as weighted training signal scaled by user-investment proxies (turn count, correction explicitness). Longitudinal user-level evaluation metrics that surface the FPG cost so it can be tracked and reduced over time.

5.7 Retry-Failure Invisibility (RFI)

Definition. Standard offline evaluation review records a failed turn but not the counterfactual that the same model would have succeeded under a standardized retry prompt. As a result, capability failures and recoverable behavioral failures are statistically conflated in evaluation pipelines.

Observed pattern. An evaluator reviewing transcripts sees a model declare an item unresolved and codes it as a capability failure. The fact that a follow-up prompt with no new information produced a correct answer is not consistently captured. Without explicit retry capture, PTS, BDTF, SMI, and VMC all manifest in transcripts as terminal failures that look like capability gaps.

Why it matters. Without a measurement that isolates retry behavior, the behavioral subset is not separable from the (much larger and more studied) capability-failure population. This is the central evaluation-pipeline blind spot this paper documents. Recoverability remains invisible as long as offline review does not run the counterfactual retry.

Differentiation. RFI is the only mode in this taxonomy whose locus is the evaluation pipeline itself, not the model or the system. It is in the taxonomy because mitigating the other modes requires first making them measurable, and that measurability is what RFI denies.

Detection. Counterfactual replay: for any flagged failed turn in offline review, replay the same prompt with explicit retry framing. If success rate jumps materially under retry, the original failure was behavioral, not capability-bound. The behavioral subset can then be re-coded against the rest of the taxonomy.

Mitigation. Augment offline evaluation pipelines with retry traces. Distinguish first-pass and retry-after-prompt success in dashboards and headline metrics. Track recoverability gap as a first-class

evaluation quantity reported alongside accuracy. This change is infrastructural rather than model-side and can be implemented today.

5.8 Stale Third-Party Tool/UI Knowledge (STK)

Definition. The model provides confident instructions for third-party tools or interfaces whose live UI has changed since training cutoff. Instructions are plausibly worded but cannot be followed because labels, menus, or setup flows differ from the model's stale knowledge.

Observed pattern. Multi-turn back-and-forth in which the user reports the live UI does not match the model's description. The model corrects only after the user describes the current UI in detail. The cost is paid as user-time spent navigating an interface the model has misdescribed, and as the user-side burden of recognizing that the model's confident instructions are stale.

Why it matters. Fast-evolving SaaS surfaces have UI-change cadences shorter than typical training-cutoff lags. The cost of stale UI knowledge is real and is paid disproportionately on developer-facing tools whose menus and labels evolve quickly. Unlike factual hallucination, the surface form of the answer is plausible; only by trying to follow the instructions does the user discover the staleness.

Differentiation. STK is procedural staleness; classic time-sensitive QA is factual staleness. STK is distinct from PTS (which is about effort allocation on a single retrieval) and from QI (which is about internal numerical contradiction). The user-visible signal is the same as a hallucination, but the locus is staleness, not invention.

Detection. Confidence-decay scoring: any output containing third-party-tool navigation instructions should be wrapped with explicit staleness language proportional to the elapsed time since training cutoff weighted by the target service's historical UI-change cadence. A registry of high-churn services makes this practical.

Mitigation. Default verification framing on tool instructions: present the path as an estimate, suggest the user confirm against current documentation, and offer to update if the user describes the live UI. The framing converts STK from a confident failure into a collaborative verification step.

5.9 Quantitative Inconsistency (QI)

Definition. Within a single response, the model emits two or more numerical claims that contradict each other under the response's own stated assumptions. The contradiction is internally checkable but is not self-audited before delivery.

Observed pattern. An analytic response contains a breakeven figure and an average-cost figure that cannot both be true under the response's own stated unit cost. Users with quantitative fluency catch the inconsistency; users without may rely on the inconsistent analysis. The model owns the error when prompted, but the absence of any self-audit before returning is the behavioral failure.

Why it matters. QI is especially costly because the answer looks analytical and authoritative. Internal contradiction can propagate into downstream artifacts (slides, spreadsheets, business plans) where it is harder to detect after the fact. The user-visible signal is conscientiousness; the underlying behavior is the absence of arithmetic verification.

Differentiation. QI is reasoning/self-audit-side. It is distinct from PTS (retrieval-side) and SVD (source-grounding-side). QI does not require any external information to detect; it requires intra-response arithmetic verification under the response's own stated assumptions.

Detection. Intra-response consistency checks: programmatically extract numerical claims, their stated relationships, and verify arithmetic consistency under the response's own assumptions. The check is

mechanical and cheap; it can run model-side as a self-audit or system-side as a post-pass.

Mitigation. An explicit numeric self-audit pass before returning responses with two or more quantitative claims. Where the audit fails, the model either revises or surfaces the inconsistency to the user. The audit is computationally trivial relative to the generation that produced the claims.

5.10 Phantom Placeholder Persistence (PPG)

Definition. The model issues a command, file path, citation field, configuration value, or code snippet containing an unresolved template placeholder even when the surrounding context implies a concrete value is needed. The placeholder substring is emitted as if it were a final value.

Observed pattern. A shell command emitted with a literal placeholder substring; a citation block with a template author or year field; a code snippet with a generic project path. The user pastes the command and receives an immediate error. The fix is mechanical—substitute the context-supplied value—but the responsibility was silently shifted to the user.

Why it matters. PPG silently shifts work to the user. Placeholders that should have been resolved from context are emitted as final output with no warning that substitution is required. The cost is small per instance but is paid every time the user runs the command and discovers the failure on execution.

Differentiation. PPG is artifact-side, not reasoning-side. It does not require the model to know any additional facts; it requires the model to recognize when emitted output is meant to be runnable versus illustrative, and to substitute or warn accordingly.

Detection. Regex-detect placeholder patterns in code-block output: angle-bracketed tokens, path-template strings such as ‘path/to’ or ‘your-’, and common template names. Each match should trigger either substitution from session context or an explicit user-substitution prompt.

Mitigation. A pre-delivery scan of code and command outputs for placeholder patterns. When found, substitute from session context if context unambiguously supplies the value; otherwise mark the placeholder explicitly and prompt the user. The check is deterministic and cheap.

5.11 Usage-Cap Termination (UCT)

Definition. A rate limit or usage cap terminates the conversation mid-task. The system surfaces a user-facing message but recovery often degrades to a malformed completion rather than graceful state preservation. The continuation turn following the cap event does not reliably resume the task.

Observed pattern. After a long agentic stretch, the model emits a usage-cap notice. Subsequent continuation prompts elicit a short, non-substantive completion that does not resume the task. The pattern can recur within a single session under sufficient agentic load, compounding workflow disruption.

Why it matters. UCT is system-level, but its user-visible effect is workflow-integrity loss. A failure to resume cleanly turns a rate-limit event into a workflow disruption disproportionate to the actual cap. The user loses not only the throttled tokens but also the continuity that depended on resuming where the session paused.

Differentiation. UCT is rate-limit-driven; MRT is budget-driven within a single response; MSI is client/transport-driven. The mitigation surfaces differ—graceful resume design for UCT, structured progress for MRT, client buffering for MSI—but the user-visible appearance can be similar.

Detection. System telemetry on conversation-level cap events and on the success rate of the subsequent continuation turn. The latter is the resume-state quality metric; a high rate of malformed continuations indicates degraded UCT recovery.

Mitigation. Graceful resume design: when a cap fires, the system should preserve a structured progress ledger, summarize work completed, and provide a clean resume entry point when the cap clears. The continuation turn should never degrade to a non-substantive completion such as a literal no-response token.

5.12 Visual Misreading Cascade (VMC)

Definition. Given a structured visual input (chart, table, dashboard, form, schematic, or culturally conventional diagram), the model produces a confident interpretation. When the user corrects a specific structural element, the model produces a new interpretation that is itself incorrect in a different way. The cascade can repeat across multiple correction cycles.

Observed pattern. Sequential confident misreadings of basic structural elements of a structured visual input. Each user correction triggers a fresh interpretation rather than a paused structural confirmation; new errors substitute for old ones until the user explicitly restates the structure. The model never reaches a state where it asks the user to verify chart structure before committing to interpretation.

Why it matters. Multimodal interpretation of structured symbolic inputs (charts, financial statements, diagrams, forms) is a growth area for LLM workflows. VMC implies that the model is not performing a stable first-pass structural extraction before downstream reasoning. The cost compounds because downstream analysis depends on the misread structure.

Differentiation. VMC is distinct from PTS (model does not decline; it produces confident output), from STK (input is user-supplied, not a third-party UI), and from QI (each interpretation is internally consistent; the failure is at the visual-input to symbolic-interpretation boundary).

Detection. Visual-input verification loop: on first interpretation of a structured visual input, the model emits a compact structural summary (field-by-field, line-by-line, cell-by-cell) and explicitly requests confirmation before producing downstream analysis.

Mitigation. Pause-and-confirm protocol on structured visual inputs. The cost is one round-trip; the benefit is elimination of the cascade. Implementable as a model-side reflex or a system-level multimodal pre-check before downstream analysis depends on the reading.

5.13 Artifact Finalization Failure (AFF)

Definition. In artifact-producing workflows, the model generates a deliverable (PDF, DOCX, slide deck, spreadsheet, code file) that nominally exists but does not satisfy user-facing readiness criteria. Terminal text quality is acceptable; artifact-level readiness is not.

Observed pattern. A generated document opens but has misaligned headings, broken links, missing sections, or layout problems that make it unfit for the stated purpose. A generated code file runs but has minor formatting that the user must repair before sharing. The user discovers the failure only on opening or running the artifact.

Why it matters. Terminal text quality is necessary but not sufficient for artifact tasks. Many professional workflows end in deliverables whose acceptance criteria are visual or structural rather than purely textual. A model that produces excellent text but a flawed file imposes recovery cost equal to the gap, and the gap is invisible until the user opens the artifact.

Differentiation. AFF is artifact-readiness-side; PPG concerns specific placeholder substrings inside an otherwise-formatted output; EBC concerns boundary leakage of advisory content. All three can co-occur in a single artifact, and the rubric distinguishes them by what specifically fails the readiness check.

Detection. Automated artifact checks: file opens, page count matches expectation, headings render, no placeholder tokens remain, links resolve, citations support claims, format-specific lints pass. Human review supplements where mechanical checks are insufficient.

Mitigation. A pre-delivery artifact-readiness audit: deterministic checks before emitting the file. Where checks fail, regenerate or surface the failure. Treat the artifact as the unit of evaluation rather than the surrounding text; this is a measurement discipline as much as a model-side change.

5.14 Source Verification Drift (SVD)

Definition. A workflow begins with a requirement to verify sources, claims, citations, or factual assertions, but progressively drifts into plausible synthesis without sufficient source grounding. The drift can be subtle: citations may support adjacent points but not the exact claim, or a source may be stale for a time-sensitive assertion.

Observed pattern. Cited sources support adjacent points but not the exact claim. Citation freshness slips on time-sensitive assertions. The model continues to attach citations to claims that are now partially synthesized rather than directly supported. The presence of citations creates a surface appearance of grounding even when the grounding has weakened.

Why it matters. SVD is corrosive because citations create the appearance of grounding even when grounding has weakened. Downstream consumers of the artifact (reviewers, readers, decision-makers) cannot easily distinguish well-supported claims from drifted ones without re-verifying every citation.

Differentiation. SVD is citation/retrieval-discipline-side. It differs from classical hallucination (which fabricates the claim itself) in that the claim is plausible and the citation is real, but the alignment between them has weakened. It is distinct from QI, which concerns internal arithmetic, and from STK, which concerns stale procedural knowledge.

Detection. Claim-source entailment checks for each cited claim. Link-freshness audits for time-sensitive assertions. Independent verification on a sample of claims, with results compared to the entailment check; large divergence indicates drift.

Mitigation. An explicit verification pass before delivering citation-heavy artifacts: each claim is matched to a source span, and the citation is retained only if the match holds. Where entailment is partial, the claim is softened or removed.

5.15 Editorial Boundary Contamination (EBC)

Definition. Process commentary, advisory metadata, scoring or readiness language, venue strategy, or other chat-level material appears inside a deliverable that should be publication-ready. The artifact-level boundary between conversation and final output fails.

Observed pattern. An intermediate manuscript draft contains a sentence about its own venue suitability. A public summary contains an instruction the user had previously issued in chat. A submitted artifact contains a meta-note about what changed between versions. The user must audit deliverables for advisory content even after substantive content is correct.

Why it matters. Boundary contamination is recoverable but costly because it bleeds private process into public output. The reputational and privacy cost can outweigh the substantive value of the artifact. In professional settings the leakage can be material.

Differentiation. EBC is the artifact-side analogue of the more general distinction between advisory dialogue and final artifact. It is distinct from AFF (which concerns format and readiness) and from SVD (which

concerns source grounding). EBC is specifically about boundary management between conversation and deliverable.

Detection. Pre-delivery boundary audit: scan the artifact for explicit advisory language (readiness scores, venue lists, ‘you should,’ ‘I recommend,’ process explanations) and for content that should have remained in chat.

Mitigation. An explicit dual-channel protocol: conversation receives advisory content; the artifact receives only content the user asked to be in the artifact. Pre-delivery audit applied automatically to publication-style outputs.

5.16 Audience / Context Miscalibration (ACM)

Definition. The model produces technically correct content that is poorly calibrated for the intended audience, register, or distribution channel. The substantive content is acceptable; the communicative fitness for the audience or channel is not.

Observed pattern. A public-communication artifact contains identifying or personal context that should not appear publicly. A summary written for a general audience uses technical terminology better suited to a specialist audience. A short executive update is delivered as a long, citation-heavy academic paragraph.

Why it matters. ACM is the bridge between substantive content quality and communicative fitness. A model that produces correct content for the wrong audience imposes rework on the user and, in public-communication contexts, can introduce reputational or privacy risk through accidental disclosure.

Differentiation. ACM is audience-side; AFF is format-side; EBC is process-leakage-side. ACM can co-occur with EBC when private process notes leak into an artifact aimed at a public audience, but the two failures are conceptually distinct.

Detection. Audience-and-channel checks: items in the pilot pack require that a summary avoid sensitive personal-context terms and avoid naming specific vendors in audience-restricted artifacts. Boundary terms include identifying personal context, employer-sensitive content, and unrelated personal subject matter.

Mitigation. Explicit audience and channel framing in the prompt or system policy. A pre-delivery audit that checks for terms incompatible with the stated audience, similar in spirit to the EBC audit. The framing should be set by the user, surfaced in the system policy, and enforced by the audit.

6. Cross-Workflow Observations

6.1 Document-generation and artifact-production workflows

When the deliverable is a document, slide deck, spreadsheet, or code file, terminal text quality is necessary but not sufficient. The artifact must open cleanly, render correctly, and meet the user's apply-ready criteria. AFF, PPG, and EBC are concentrated in these workflows; their incidence becomes visible only when the artifact is evaluated outside the chat. A reasonable evaluation discipline treats the artifact, not the response describing the artifact, as the primary unit of measurement.

6.2 Source-verification and multi-step research workflows

Citation-heavy and verification-heavy workflows surface SVD, PTS, and SMI together. The model may begin with disciplined source-grounding, drift into plausible synthesis as the task lengthens, and selectively escalate retrieval on some claims but not others. Claim-level verification checks are the appropriate measurement; citation presence alone is insufficient.

6.3 Public-communication and audience-targeted workflows

When an artifact is intended for a public or restricted audience, ACM and EBC become salient. Technically correct content can leak private context, off-register tone, or advisory metadata. Mitigation is policy-level rather than model-capability-level: explicit audience framing in the prompt or system instruction, combined with a pre-delivery audit for boundary contamination.

6.4 Quantitative reasoning workflows

QI surfaces most often when responses contain multiple quantitative claims under shared assumptions. The detection method is mechanical (intra-response arithmetic check) and is currently underused. Quantitative consistency is well-suited to deterministic automated checks, which makes it an attractive first target for benchmark inclusion.

7. Pilot Experiment: Retry-Probe Quantification of PTS

7.1 Setup

The primary measured case is a fifteen-item heterogeneous retrieval task. Each item requires retrieval, interpretation, and structured per-item output. Items belong to three structural classes: (1) direct URLs with human-readable paths (n=9); (2) URLs containing opaque numeric identifiers requiring secondary lookup to extract title or content (n=3); (3) title-only references requiring search (n=3). The first pass is evaluated as successful if the model produces a substantive resolution. The retry pass is evaluated under the same criterion after a standardized user-prompted retry that supplies no new substantive information.

7.2 Procedure

For each item, two binary outcomes were recorded from the verbatim session transcript: (i) first-pass success, and (ii) success after a single standardized retry of the form 'try a different approach.' The retry prompt did not supply new documents, new tools, or new task information. Coding was conservative: an item was counted as a recoverable behavioral failure only when the second-pass success demonstrated a previously available capability rather than newly supplied information.

7.3 Results

Metric	Value	95% CI
Total items (N)	15	—
First-pass resolved (p_1)	12 / 15 = 80.0%	Wilson: [54.8%, 93.0%]
Resolved after retry (p_r)	15 / 15 = 100.0%	Wilson: [79.6%, 100.0%]
Recoverable first-pass failures	3 / 15 = 20.0%	Wilson: [7.0%, 45.2%]
Recoverability gap (p_r - p_1)	+20.0 pp	Discordant-proportion CI: [7.0%, 45.2%]

Table 4. Retry-probe pilot results with Wilson 95% confidence intervals.

Item structural class	n	First-pass resolved	First-pass rate	Recoverable failures
Direct URL (human-readable path)	9	9	100%	0
URL with opaque numeric identifier	3	0	0%	3
Title-only (search-required)	3	3	100%	0

Table 5. Pilot first-pass performance by item structural class.

7.4 Exact small-sample statistical analysis

The recoverable-failure rate is 3 / 15 = 20.0%, with Wilson 95% confidence interval [7.0%, 45.2%]. The width of the interval reflects the small sample. The structural-class association is striking: 0 of 3 opaque-identifier items resolved on first pass; 12 of 12 non-opaque items resolved on first pass.

	First-pass success	First-pass failure	Row total
Opaque-identifier	0	3	3
Non-opaque	12	0	12
Column total	12	3	15

Table 6. Contingency table for first-pass success by item structural class.

A two-sided Fisher exact test on the 2 by 2 contingency table yields $p = 0.0022$. The one-sided test for the hypothesis that opaque-identifier items have lower first-pass success also yields $p = 0.0022$. The result is statistically meaningful for this item set but should not be read as a population-level rate. The item set was not randomly sampled from a defined universe; it is a single heterogeneous batch from a naturalistic workflow.

7.5 What the pilot supports and does not support

- It supports the **existence** of recoverable behavioral failures in at least one measured retrieval workflow.
- It supports treating first-pass and retry-after-prompt success as separately measurable quantities.
- It suggests, exploratorily, that opaque numeric identifiers may be a high-incidence trigger for PTS under batch load.
- It does **not** establish a population-level PTS rate for any model or vendor.
- It does **not** establish a universal opaque-identifier-to-failure relationship.
- It does **not** establish a model ranking, because workload and model identity are not experimentally balanced.

8. Observational Notes Beyond the Pilot

Beyond the measured PTS pilot, the case-study corpus contains transcript-verifiable instances of additional modes. These are reported at the level of pattern, trigger, and recovery so that they can be probed in controlled replication. No additional rates are claimed for these modes; they are Tier 2 observations supporting taxonomy and protocol design rather than quantitative results. The corpus includes instances of STK (multi-turn navigation of a third-party developer console with stale UI descriptions), QI (single-response analytic output with internally inconsistent numeric claims), PPG (shell commands containing literal placeholders the surrounding context resolved), VMC (sequential confident misreadings of a structured visual document), UCT (cap-driven terminations followed by malformed continuation completions), and BDTF (uniform shallow per-item dispositions under multi-item batches).

9. Proposed Multi-Model Evaluation Protocol

This section specifies a controlled multi-model benchmark designed to break the model-task confounds described in §4.4 and to test the modes catalogued in §5. The protocol is grounded in an accompanying ten-task evaluation pack covering five task categories. Results from running this protocol are not yet available; the protocol is described as a **planned replication**, not a completed study.

9.1 Task families

Family	Modes targeted	Example measurement
Retrieval retry probes	PTS, SMI	First-pass vs. retry success on direct, opaque-ID, and title-only items
Batch retrieval and scaling	BDTF	Per-item correctness as batch size increases from 1 to 30
Quantitative consistency	QI	Internal arithmetic consistency in responses with multiple numerical claims
Structured visual interpretation	VMC	Field-extraction accuracy before downstream interpretation

Family	Modes targeted	Example measurement
Third-party UI / tool instructions	STK	Accuracy and staleness framing for fast-changing tool instructions
Phantom placeholders	PPG	Placeholder leakage rate in code and command outputs
Artifact finalization and delivery	AFF, PPG, EBC	Downloadability, formatting, placeholder absence, boundary cleanliness
Editorial boundary contamination	EBC	Absence of process commentary in publishable artifacts
Audience / context calibration	ACM	Absence of off-register or boundary-inappropriate content

Table 7. Task families in the proposed multi-model evaluation protocol.

9.2 Model coverage

The protocol is designed for parallel administration across frontier model families. Concrete targets include but are not limited to the Claude family (Sonnet, Opus), the GPT family, the Gemini family, and any additional frontier model an investigator can access. The pilot pack scaffolding supports the three named API surfaces and is extensible. The objective is to administer matched tasks under standardized prompts, retry prompts, tool access, and decoding settings, so that any observed differences across models are attributable to model identity rather than to task or prompt heterogeneity.

9.3 Metrics

Metric	Definition	Applies to
First-pass success	Proportion solved without retry	All task families
Retry success	Proportion solved after standardized retry	All recoverability probes
Recoverability gap	Retry success minus first-pass success	PTS, BDTF, VMC, AFF
Artifact-readiness rate	Deliverables passing objective file and formatting checks	PDF / DOCX / slides / spreadsheets / code
Numeric consistency rate	Proportion of multi-numeric responses passing intra-response audit	Quantitative tasks
Placeholder leakage rate	Proportion of code/command outputs containing unresolved placeholders	Code and command-generation tasks
Boundary-contamination rate	Artifacts containing process notes, ratings, or chat-only content	Publication and public-communication artifacts
Verification compliance rate	Proportion of items where stated verification was performed	Verification-requiring tasks
Claim-source support rate	Claims supported by cited sources at the required specificity	Research and citation tasks
Human intervention cost	Number of corrective turns to reach acceptable output	All workflow tasks

Table 8. Metrics for the proposed multi-model evaluation protocol.

9.4 Procedure

1. **Item construction.** For retrieval, construct a balanced item set: 20 direct URLs, 20 opaque-identifier URLs, 20 title-only references.

2. **Prompt standardization.** Each model receives the same initial prompt and, for recoverability probes, the same standardized retry prompt.

3. **Tool and setting standardization.** Tool access, temperature, max-tokens, and system-prompt content held constant across models within a family of items.

4. **Batch design.** Retrieval items administered at N = 1, 5, 15, 30 to support batch-scaling analysis.

5. **Annotation.** Each item double-coded by independent annotators. Inter-annotator agreement reported using Cohen's kappa or Krippendorff's alpha.

6. **Reporting.** Confidence intervals for all rates. Exact tests for small-sample comparisons.

9.5 Sample size and power

The pilot's N = 15 yields wide confidence intervals. A benchmark intended to estimate recoverability rates with narrower intervals should size each condition accordingly. For a binary failure rate near 20% with desired margin of error +/- 5 percentage points at 95% confidence, roughly 250 items per condition are required; for +/- 3 points roughly 700; for +/- 2 points roughly 1500. Detection of a 10-percentage-point across-model difference at 80% power requires approximately 200 items per arm under comparable rates.

10. Mitigation Implications

The taxonomy suggests practical, low-cost mitigations that do not require new model capabilities. They require improved execution policy, product affordances, and evaluation criteria. The mitigations below map directly to the modes in §5.

- **Explicit escalation policy** for retrieval (PTS, SMI).
- **Batch-aware effort budgeting** with per-item verification flags (BDTF).
- **Self-verification gates**: arithmetic checks (QI), claim-source entailment (SVD), file-readiness checks (AFF, PPG).
- **Visual-input verification loop** (VMC).
- **Staleness-aware tool guidance** (STK).
- **Pre-delivery boundary audit** (EBC); audience-and-channel checks (ACM).
- **Graceful resume design** across interruptions (MRT, UCT, MSI).
- **Counterfactual-replay evaluation** (RFI).

11. Limitations and Threats to Validity

1. **Single-user evidence base.** Fourteen sessions from a single user. Generalization requires multi-user replication.

2. **Single-coder bias.** Future work includes a double-coded reliability subsample with reported Cohen's kappa.

3. **Inductive taxonomy.** Sixteen modes may be incomplete; some may collapse empirically (see §5.0 for rationale).

4. **Model self-report uncertainty.** Mechanism descriptions that rest on model self-reports are treated as suggestive.

5. **Missing per-message model attribution** in the Claude.ai export schema.

6. **Model-task confound.** Case-study task type and model identity are partially correlated; the multi-model protocol is the appropriate vehicle for clean comparison.

7. **Single-batch quantitative anchor.** Section 7 reports one batch with wide confidence intervals.

8. **Selective observability.** Failures are easier to detect when users are expert enough to challenge outputs.

9. **Cross-platform abstractions are summary-level.** Replication should include explicit task logs.

12. Future Work

1. Run the §9 multi-model protocol across Claude, GPT, Gemini, and additional frontier model families.

2. Expand to a multi-user corpus with double-coding and reported kappa.

3. Longitudinal tracking across major model-version releases.

4. Artifact-evaluation infrastructure for downloadability, formatting, placeholder absence, link freshness, boundary contamination.

5. Recommendation to AI providers: include a model field on each message in conversation-export schemas.

6. Behavioral-failure capture in product feedback flows.

13. Conclusion

Recoverable behavioral failures are cases where a model appears capable of completing a task but fails to exercise that capability on the first pass. They are not captured by standard terminal-output evaluation, because they become visible only when a retry, correction, or artifact audit reveals that the model could have produced the correct output without new substantive information. This paper contributes a definition, a layered sixteen-mode taxonomy, a quantitative retry-probe pilot with exploratory confidence intervals and an exact small-sample association test, abstracted cross-platform observations of artifact-readiness and editorial-boundary modes, and a multi-model evaluation protocol grounded in a ten-task pilot pack and supporting analysis scripts. The central implication is that AI evaluation should measure not only capability but also persistence, verification, escalation, artifact readiness, and boundary control.

References

- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., et al. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv:2204.05862*.

- Chen, W., Wang, X., & Wang, W. Y. (2021). A dataset for answering time-sensitive questions. *arXiv:2108.06314*.

- Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. *NeurIPS*.

- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). Measuring massive multitask language understanding (MMLU). *ICLR*.

• Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*.

• Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., et al. (2022). Holistic evaluation of language models (HELM). *arXiv:2211.09110*.

• Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. *ACL*.

• Liska, A., Kocisky, T., Gribovskaya, E., Terzi, T., Sezener, E., Agrawal, D., et al. (2022). StreamingQA: A benchmark for adaptation to new knowledge over time. *ICML*.

• Park, P. S., Goldstein, S., O'Gara, A., Chen, M., & Hendrycks, D. (2024). AI deception: A survey of examples, risks, and potential solutions. *Patterns*, 5(5).

• Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., et al. (2023). Towards understanding sycophancy in language models. *arXiv:2310.13548*.

• Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., et al. (2023). Beyond the imitation game (BIG-Bench). *TMLR*.

• Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.). SAGE.

• Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., et al. (2024). WebArena: A realistic web environment for building autonomous agents. *ICLR*.

Appendix A. Observation Bank Coding Schema

Field	Description
observation_id	Stable identifier
task_family	Retrieval, artifact, visual, quantitative, UI/tool, source-grounding, other
original_prompt	User request (abstracted where needed for privacy)
first_pass_output	Model response or artifact outcome on the first attempt
failure_criterion	Reason the first-pass output is judged a failure
retry_prompt	Standardized retry used to test recoverability
new_information_introduced	Yes / No; if Yes, item is excluded from RBF
post_retry_output	Second-pass result
mode_code	One of the 16 codes in Table 3
trigger	Observed condition under which the failure occurred
recovery_type	Search escalation / verification / structural confirmation / boundary audit / other
user_visible_cost	Time, confusion, unusable artifact, missed information, rework
evidence_tier	Tier 1 / 2 / 3 / 4 per §4.2

Table A1. Observation-bank coding schema.

Appendix B. Experiment Pack and Task Schema

The accompanying experiment pack provides a small pilot benchmark and runner scaffold for measuring recoverable-behavioral-failure proxies across multiple LLM APIs. It contains ten items distributed across five categories, plus runner and scorer scripts that administer the items against OpenAI, Anthropic, and Google API surfaces.

Category	n	Purpose
QI	2	Internal numerical consistency on responses with multiple quantitative claims
PPG	2	Placeholder leakage in code or shell-command outputs
EBC	2	Absence of process commentary in publication-style outputs
ACM	2	Audience appropriateness in public-summary outputs
RETRY_PROXY	2	Stated retry strategy for retrievals returning thin or empty results

Table B1. Categories in the v1.0 experiment pack.

Appendix C. Annotation Rubric

1. Did the first-pass output satisfy the task's acceptance criteria? If yes, label *first-pass success*.
2. If the first-pass output failed, did the retry succeed?
3. Did the retry prompt introduce new substantive information? If yes, label *ordinary interactive correction*.

4. If the retry succeeded without new information, label *recoverable behavioral failure* and assign a mode code.

5. If the retry also failed, label *non-recoverable under available context*.

6. If determination is unclear, label *ambiguous*.

Appendix D. Reproducibility Checklist

- Item set with stable IDs, task-family labels, prompts, retry prompts, expected outputs, verification methods.
- Model and tool settings recorded: model name, temperature, max-tokens, system prompt, tool list, decoding settings.
- Raw outputs archived in append-only JSONL with timestamps.
- Automated checks distributed as a separate scorer script.
- Annotation guidelines and double-coded labels released with kappa or alpha reported.
- Analysis notebook producing all tables, confidence intervals, statistical tests.
- Data statement describing collection method, privacy controls, redaction procedures, licenses.
- Versioning: pack version, model versions, analysis-notebook commit hash recorded in the results file.